

AUTOM-96 LAB: GitHub Actions CI/CD for Dynatrace Terraform

Series: AUTOM — Dynatrace Automation | **Reference:** 96 — GitHub Actions CI/CD LAB | **Created:** May 2026 | **Last Updated:** 05/13/2026

Overview

Hands-on lab that takes the GitHub Actions material from **AUTOM-07: CI/CD Integration** §3 and the credential-handling pattern from **AUTOM-04: Terraform Provider** §3 and walks through them end-to-end. By the end of this lab you have a working pipeline that:

- Runs `terraform fmt -check`, `terraform validate`, and `terraform plan` on every pull request, posting the plan summary back as a PR comment.
- Runs `terraform apply` on merge to `main`, gated by a GitHub Environment with required reviewers.
- Authenticates to Dynatrace via **GitHub OIDC -> AWS IAM role -> HashiCorp Vault -> runtime fetch** of a Dynatrace Platform Token. **No static Dynatrace token lives in GitHub Secrets.**
- Stores Terraform state in S3 with native S3 lockfile locking, encrypted at rest.
- Manages a single Settings 2.0 resource (Kubernetes generic-metadata enrichment) end-to-end so the credential plumbing is the focus, not the resource model.

This LAB is the hands-on companion to AUTOM-07 §3 (which describes the patterns) and AUTOM-04 §3 (which describes the token model). It does **not** repeat material from those notebooks — it walks through the steps.

Out of scope for this LAB (cross-references provided):

Topic	See
Other CI platforms (GitLab, Bitbucket, Bamboo, Azure DevOps)	AUTOM-07 §4–§7
State backend hardening (KMS, IAM scopes, multi-account)	AUTOM-09 §3

Topic	See
Multi-environment promotion patterns	AUTOM-09 §4
Lifecycle protections (<code>prevent_destroy</code> , <code>replace_triggered_by</code>)	AUTOM-09 §9
Token-model deep dive (Platform vs classic API)	AUTOM-04 §3
<code>dynatrace_api_token</code> / state-leakage concerns	AUTOM-04 §3

Table of Contents

1. [Prerequisites](#)
2. [Architecture](#)
3. [Step 1 — Bootstrap the Terraform repo layout](#)
4. [Step 2 — Mint the Platform Token \(out-of-band\)](#)
5. [Step 3 — Wire OIDC trust](#)
6. [Step 4 — Author the workflow file](#)
7. [Step 5 — Run a plan-only test on a PR](#)
8. [Step 6 — Promote to apply on merge](#)
9. [Step 7 — Validate](#)
10. [Common Pitfalls](#)
11. [Where to go next](#)
12. [Summary Checklist](#)
13. [Sources and References](#)

1. Prerequisites

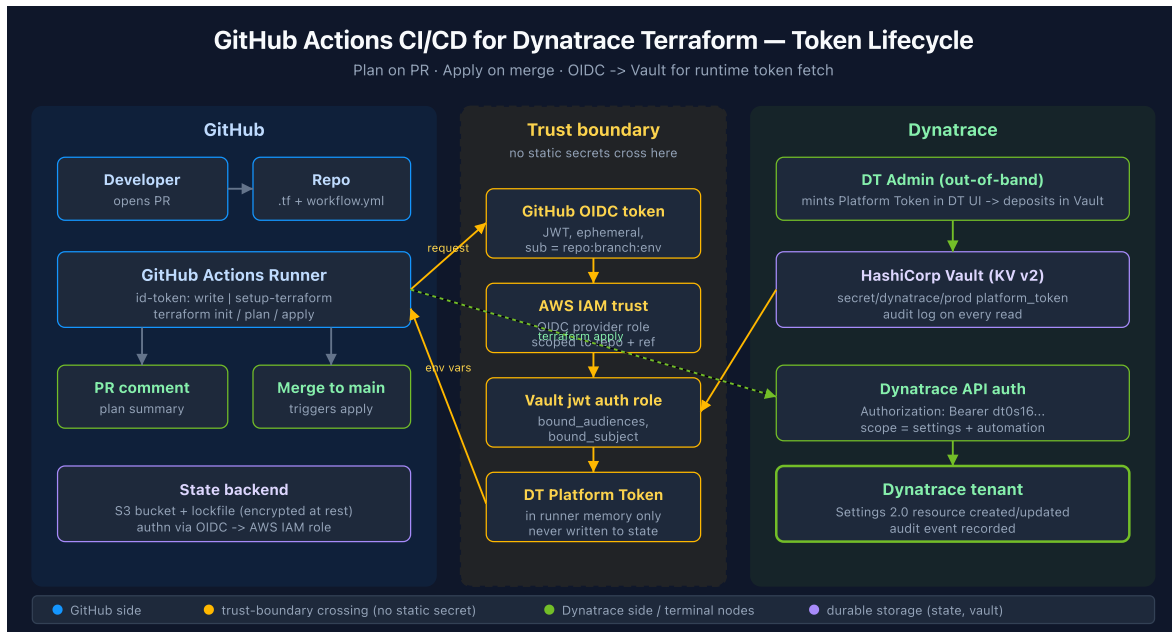
Requirement	Details
Completed	AUTOM-04 (Terraform Provider) and AUTOM-07 (CI/CD Integration) §3 read at least once

Requirement	Details
Dynatrace Environment	Gen3 SaaS tenant; account-admin access to mint a Platform Token on a service user
GitHub Account	Repository with admin access (to add Environments, configure OIDC permissions, set repo secrets)
AWS Account	Account where you can create an IAM OIDC identity provider, an IAM role, an S3 bucket, and (optionally) a KMS key. A Free Tier account works.
HashiCorp Vault	An instance reachable from GitHub Actions. HCP Vault Free (managed, free tier) works; a self-hosted Vault behind a public endpoint or a GitHub-hosted-runner-reachable VPN endpoint also works.
Local tools	<code>terraform >= 1.6</code> , <code>aws CLI</code> , <code>vault CLI</code> , <code>gh CLI</code> (GitHub CLI), <code>git</code>
Time budget	Roughly 1.5–2 hours end-to-end, the first time you run it.

Tenant URL placeholder. Throughout this LAB, replace `https://<your-tenant>.apps.dynatrace.com` (UI host) and `https://<your-tenant>.live.dynatrace.com` (API host) with your actual tenant identifier.

2. Architecture

The pipeline has three zones: **GitHub** (source + runner + state), a **trust boundary** crossed by short-lived federated identity (no static secrets), and **Dynatrace** (the Platform Token mint side and the eventual API target).



Why this shape

- **Mint out-of-band, deliver in-band** (the pattern from AUTOM-04 §3): the long-lived Dynatrace credential is generated once by an admin, deposited in Vault, and never touched by Terraform or by GitHub Secrets.
- **OIDC federation** removes the chicken-and-egg of "what authenticates the credential fetch?" — GitHub's OIDC identity authenticates to AWS IAM and to Vault directly, so there is no static AWS access key or static Vault token in GitHub Secrets either.
- **Single id-token: write** permission on the workflow is the only GitHub-side configuration that matters; the rest is set up once on the AWS and Vault sides.

3. Step 1 — Bootstrap the Terraform repo layout

The LAB uses the simplest layout that exercises the credential plumbing — one directory, three `.tf` files, one Settings 2.0 resource. For multi-environment / module-based layouts see **AUTOM-09 §2**.

Create the directory and files

```
mkdir -p dynatrace-terraform/.github/workflows
cd dynatrace-terraform
git init
touch versions.tf main.tf backend.tf .gitignore
```

`.gitignore` (mandatory before first commit)

```
# Terraform
.terraform/
.terraform.lock.hcl
*.tfstate
*.tfstate.*
*.tfplan
crash.log

# Local credentials
*.tfvars
!example.tfvars
.env
```

Why this matters. State files and `.tfvars` files routinely contain sensitive values — even with `sensitive = true` they are still plain-text on disk (see AUTOM-04 §3 *Operational Safety*). Committing them to Git is the most common credential-leak path in real Terraform shops.

`versions.tf` — pin everything

```
terraform {
  required_version = ">= 1.6.0"

  required_providers {
    dynatrace = {
      source  = "dynatrace-oss/dynatrace"
      version = "~> 1.93"
    }
  }
}
```

backend.tf — S3 state, native S3 lockfile

```
terraform {
  backend "s3" {
    bucket      = "my-org-terraform-state"
    key         = "dynatrace/lab/terraform.tfstate"
    region      = "us-east-1"

    # Native S3 lockfile (Terraform 1.10+) – replaces the old DynamoDB lock
    # table.
    # See https://developer.hashicorp.com/terraform/language/backend/s3 for
    # the
    # current locking model.
    use_lockfile = true

    encrypt      = true
  }
}
```

Bucket setup is out of scope for this LAB. Create the bucket once via console / CloudFormation / a separate Terraform configuration; enable bucket versioning; consider an SSE-KMS key. See **AUTOM-09 §3** for the hardening checklist.

main.tf — one Settings 2.0 resource

We pick **Kubernetes generic metadata enrichment** because it (a) is a Settings 2.0 schema, so it works with a Platform Token alone, (b) is harmless to create and delete in a non-prod tenant, and (c) is unlikely to collide with an existing rule.

```

provider "dynatrace" {
  # Picked up from environment variables at apply time:
  #   DYNATRACE_ENV_URL
  #   DYNATRACE_PLATFORM_TOKEN
}

resource "dynatrace_generic_setting" "lab_k8s_team_label" {
  schema_id = "builtin:kubernetes.generic.metadata.enrichment"
  scope      = "environment"

  value = jsonencode({
    enabled = true
    rules = [{
      sourceAttribute = "namespace-label"
      sourceKey        = "lab-team"
      targetAttribute = "dt.security_context"
    }]
  })
}

```

No `dt_platform_token` argument in the provider block. The Dynatrace provider reads `DYNATRACE_ENV_URL` and `DYNATRACE_PLATFORM_TOKEN` from the environment when the corresponding HCL arguments are absent. This is what lets the OIDC -> Vault -> env-vars chain work without HCL-level coupling.

4. Step 2 — Mint the Platform Token (out-of-band)

This step happens **once**, in the Dynatrace UI, by a human admin. It is deliberately not part of the pipeline. The full rationale (mint-out-of-band-vs-in-pipeline trade-offs) is in **AUTOM-04 §3 Operational Safety — State File Leakage** — read that subsection if you have not already.

UI steps

1. Sign in to `https://<your-tenant>.apps.dynatrace.com` as an account administrator.
2. Open the **Account Management** console (the user-icon menu in the top right) -> **Identity & access management** -> **Service users**.
3. Create or select the service user that will own this token (e.g. `svc-terraform-lab`).

4. Confirm the service user is in a group with the IAM permissions needed for Settings 2.0 management — at minimum the equivalent of "**Change monitoring settings**" (`environment:roles:manage-settings`) on the tenant.
5. Switch to **Platform tokens** (myaccount.dynatrace.com -> Platform tokens) and click **Generate new token**.
6. **On behalf of:** select the `svc-terraform-lab` service user.
7. **Scopes:** select `settings:objects:read` and `settings:objects:write` . (Add `settings:schemas:read` if your tenant requires it for schema discovery.)
8. Set an **expiration** appropriate to your rotation policy (the LAB recommendation is 90 days — short enough to force you through the rotation flow at least once during the LAB lifetime).
9. Copy the token value (`dt0s16....`). It is shown **once**.

Deposit the token into Vault

```
# Authenticate to Vault as a human admin (any auth method that works for you)
vault login -method=oidc

# Write the token to the path the workflow will read from
vault kv put secret/dynatrace/lab \
  platform_token="dt0s16.XXXX..." \
  env_url="https://&lt;your-tenant&gt;.live.dynatrace.com"
```

Three things must align for the Platform Token to actually work — service user IAM permissions, the creator's `iam:service-users:use` permission, and the scopes selected at token creation. AUTOM-04 §3 *Service User Credentials for Terraform* covers the model in full. If `terraform apply` later fails with a 403, this is the first place to look.

5. Step 3 — Wire OIDC trust

Two trust relationships need to exist before the workflow can run:

1. **GitHub -> AWS** — so that the workflow can assume an AWS IAM role to access S3 (state) without a static AWS access key.

2. **AWS -> Vault** — so that the workflow, having assumed an AWS role, can authenticate to Vault and pull the Dynatrace Platform Token.

Equivalently you can do **GitHub -> Vault directly** via Vault's `jwt` auth method bound to GitHub's OIDC issuer, skipping the AWS hop entirely. Both shapes are recommended in the [GitHub OIDC docs \(GitHub docs\)](#). This LAB uses the AWS hop because it (a) is the most common shape in enterprises and (b) gets you AWS access for the state backend in the same step.

5.1 — Create the GitHub OIDC identity provider in AWS

This is a one-time, account-wide setup. If your AWS account already has the GitHub provider registered, skip this.

```
# Use the AWS account where the S3 state bucket lives
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 6938fd4d98bab03faadb97b34396831e3780aea1
```

Thumbprint note. AWS no longer strictly verifies the thumbprint when the provider URL host is `token.actions.githubusercontent.com`, but the API still requires the field. The value above is the one GitHub's docs reference; check the [Configuring OpenID Connect in AWS \(GitHub docs\)](#) page for the current guidance.

5.2 — Create the AWS IAM role the workflow assumes

Create `trust-policy.json` :

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::<your-aws-account-id>;oidc-provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com"
        },
        "StringLike": {
          "token.actions.githubusercontent.com:sub": "repo:<your-org>/<your-repo>:*"
        }
      }
    }
  ]
}
```

The `sub` claim is the load-bearing scoping mechanism. `repo:<your-org>/<your-repo>:*` lets any branch in this repo assume the role. Tighten to `repo:<your-org>/<your-repo>:ref:refs/heads/main` for the apply role; keep the wildcard for the plan role. Mismatched `sub` claims are the single most common OIDC misconfiguration — see Common Pitfalls.

Create the role and attach a least-privilege policy that allows S3 state access only:

```

aws iam create-role \
  --role-name github-actions-dynatrace-terraform \
  --assume-role-policy-document file://trust-policy.json

aws iam put-role-policy \
  --role-name github-actions-dynatrace-terraform \
  --policy-name terraform-state-access \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": ["s3:GetObject", "s3:PutObject", "s3:DeleteObject"],
        "Resource": "arn:aws:s3:::my-org-terraform-state/dynatrace/lab/*"
      },
      {
        "Effect": "Allow",
        "Action": ["s3:ListBucket"],
        "Resource": "arn:aws:s3:::my-org-terraform-state"
      }
    ]
  }'
```

5.3 — Configure Vault to trust GitHub OIDC

The cleanest path is to skip AWS and let Vault accept the GitHub OIDC JWT directly. You only need this once per Vault instance:

```
# Enable the jwt auth method (idempotent; skip if already enabled)
vault auth enable jwt

# Configure it to trust GitHub's OIDC issuer
vault write auth/jwt/config \
  oidc_discovery_url="https://token.actions.githubusercontent.com" \
  bound_issuer="https://token.actions.githubusercontent.com"

# Create a policy that grants read on the secret we wrote in Step 2
vault policy write dynatrace-lab-read - &&&'EOF'
path "secret/data/dynatrace/lab" {
  capabilities = ["read"]
}
EOF

# Create a role that is assumable by the workflow
vault write auth/jwt/role/dynatrace-terraform-lab \
  role_type="jwt" \
  user_claim="actor" \
  bound_audiences="https://github.com/<your-org>" \
  bound_claims_type="glob" \
  bound_claims='{"sub":"repo:<your-org>/<your-repo>:*"}' \
  policies="dynatrace-lab-read" \
  ttl="15m"
```

bound_audiences must match the audience the workflow requests. The `hashicorp/vault-action` step lets you specify `jwtGithubAudience`. The default for that input is `sigstore`; we override it below to `https://github.com/<your-org>`.

See the [Vault JWT auth method \(Vault docs\)](#) for the full claim-binding reference.

6. Step 4 — Author the workflow file

Two jobs in one file: `plan` runs on every pull request; `apply` runs only on push to `main` and is gated by a GitHub Environment. Both jobs share a composite step that performs the OIDC -> Vault -> Dynatrace-token fetch, so the credential plumbing exists once.

Create `.github/workflows/terraform.yml`:

```

name: Terraform – Dynatrace Lab

on:
  pull_request:
    branches: [main]
    paths:
      - "**/*.tf"
      - ".github/workflows/terraform.yml"
  push:
    branches: [main]
    paths:
      - "**/*.tf"
      - ".github/workflows/terraform.yml"

# Required for OIDC. Without id-token:write the workflow has no OIDC token to
# present.
permissions:
  contents: read
  id-token: write
  pull-requests: write

env:
  AWS_REGION: us-east-1
  TF_IN_AUTOMATION: "true"
  TF_INPUT: "false"

jobs:
  plan:
    name: terraform plan
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.10.0"
          terraform_wrapper: true # exposes step outputs we read below

      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::<your-aws-account-id>;role/github-actions-dynatrace-terraform
          aws-region: ${ env.AWS_REGION }

      - name: Fetch Dynatrace Platform Token from Vault

```

```

    id: vault
    uses: hashicorp/vault-action@v3
    with:
      url: https://vault.example.com:8200
      method: jwt
      jwtGithubAudience: https://github.com/&lt;your-org&gt;
      role: dynatrace-terraform-lab
      secrets: |
        secret/data/dynatrace/lab platform_token |
DYNATRACE_PLATFORM_TOKEN ;
        secret/data/dynatrace/lab env_url          | DYNATRACE_ENV_URL

- name: terraform fmt -check
  run: terraform fmt -check -recursive

- name: terraform init
  run: terraform init -input=false

- name: terraform validate
  run: terraform validate -no-color

- name: terraform plan
  id: plan
  run: |
    terraform plan -no-color -out=tfplan
    terraform show -no-color tfplan &gt; plan.txt
  env:
    DYNATRACE_PLATFORM_TOKEN: ${ env.DYNATRACE_PLATFORM_TOKEN }
    DYNATRACE_ENV_URL: ${ env.DYNATRACE_ENV_URL }

- name: Post plan to PR
  if: github.event_name == 'pull_request'
  uses: actions/github-script@v7
  env:
    PLAN: ${ steps.plan.outputs.stdout }
  with:
    script: |
      const fs = require('fs');
      const plan = fs.readFileSync('plan.txt', 'utf8').slice(0, 60000);
      const body = [
        '### Terraform Plan',
        '',
        '&lt;details&gt;&lt;summary&gt;Click to
expand&lt;/summary&gt;',
        '',
        '```hcl',
        plan,
        '```',

```

```

        '',
        '</details>',
    ].join('\n');
    await github.rest.issues.createComment({
        owner: context.repo.owner,
        repo: context.repo.repo,
        issue_number: context.issue.number,
        body,
    });

apply:
  name: terraform apply
  needs: plan
  if: github.ref == 'refs/heads/main' && github.event_name ==
'push'
  runs-on: ubuntu-latest
  environment: production    # gated by GitHub Environment Protection Rules
  steps:
    - uses: actions/checkout@v4

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v3
      with:
        terraform_version: "1.10.0"

    - name: Configure AWS credentials via OIDC
      uses: aws-actions/configure-aws-credentials@v4
      with:
        role-to-assume: arn:aws:iam::<your-aws-account-
id>;role/github-actions-dynatrace-terraform
        aws-region: ${ env.AWS_REGION }}

    - name: Fetch Dynatrace Platform Token from Vault
      uses: hashicorp/vault-action@v3
      with:
        url: https://vault.example.com:8200
        method: jwt
        jwtGithubAudience: https://github.com/<your-org>;
        role: dynatrace-terraform-lab
        secrets: |
          secret/data/dynatrace/lab platform_token |
DYNATRACE_PLATFORM_TOKEN ;
          secret/data/dynatrace/lab env_url          | DYNATRACE_ENV_URL

    - name: terraform init
      run: terraform init -input=false

```

```
- name: terraform apply
  run: terraform apply -auto-approve -input=false
```

What the workflow does, in order

1. **Checkout the repo** — standard.
2. **Setup Terraform** — `hashicorp/setup-terraform@v3` pins to a known version. Pin a real version, not `latest`, so plan output is reproducible across reruns.
3. **AWS credentials via OIDC** — the workflow's OIDC token is exchanged for short-lived AWS credentials by assuming the IAM role created in Step 3.2. **No static AWS access key in GitHub Secrets.**
4. **Vault token fetch** — the same OIDC token (re-issued for the Vault audience) is exchanged for a short-lived Vault token bound to `dynatrace-terraform-lab` role. **No static Vault token in GitHub Secrets.** `vault-action` automatically masks the secrets in subsequent log lines.
5. `terraform fmt -check` — fail fast on style.
6. `terraform init` — downloads the Dynatrace provider; AWS credentials from step 3 authorize state read/write.
7. `terraform validate` — schema/HCL validation.
8. `terraform plan` — generates the plan; environment variables for the Dynatrace provider come from the Vault step.
9. **PR-comment step** — only runs on `pull_request` events; posts the plan back as a comment using `actions/github-script`.
10. **Apply job** — only runs on push-to-main, only after `plan` succeeds, only after the GitHub Environment's required reviewers approve.

`workflow_dispatch` is intentionally omitted. Adding manual-trigger support is a one-line addition (`on.workflow_dispatch:` block) but it widens the surface — anyone with workflow-write permission can apply on demand. Add it explicitly if your team needs it; defaulting it off is safer.

Configure the GitHub Environment

In the GitHub UI: **Settings -> Environments -> New environment -> `production`** .
Then:

- **Required reviewers:** add at least one team member who is not the author. This is the apply gate.
- **Wait timer:** optional 5–15 minute delay before the gate can be approved (useful for last-call review).
- **Deployment branches:** restrict to `main` only.
- **Environment secrets:** none needed for this LAB — secrets come from Vault at runtime.

7. Step 5 — Run a plan-only test on a PR

```
# From your local clone of the repo
git checkout -b lab-add-k8s-enrichment
git add .
git commit -m "feat: add Settings 2.0 K8s metadata enrichment rule"
git push -u origin lab-add-k8s-enrichment

# Open the PR
gh pr create --base main --title "LAB: add K8s enrichment rule" --body "Lab
change."
```

What you should see, in order

1. The `plan` job starts immediately on PR open. Expect ~60–90 seconds total wall-clock.
2. The first run downloads the Dynatrace provider — adds about 20 seconds you won't pay on subsequent runs.
3. The Vault step prints `Reading secret from secret/data/dynatrace/lab` and the masked secret values are not echoed. If you see the actual token value in the log, **stop**: the masking is broken or you typed it into the workflow YAML by mistake.
4. The `terraform plan` step prints something like:

```
Plan: 1 to add, 0 to change, 0 to destroy.
```

5. Within 5–10 seconds of the run completing, a new comment appears on the PR titled **Terraform Plan**, containing the collapsed plan output.

What you should NOT see

- A comment with raw token values. If the plan or apply ever leaks a token into logs, treat it as a credential incident: rotate the Platform Token in DT, rotate any downstream tokens, audit recent activity.
 - A failed apply on a PR. Apply should be skipped entirely on PRs (the `if:` condition gates it). If apply runs on a PR, the `if:` is wrong.
-

8. Step 6 — Promote to apply on merge

```
# After PR review and approval
gh pr merge --squash --delete-branch
```

What you should see, in order

1. A new workflow run starts on `main`, triggered by the merge commit.
2. The `plan` job runs again (and should produce the same plan as the PR-time run — see Common Pitfalls if it doesn't).
3. The `apply` job appears as **Waiting** with a *Review deployments* button — this is the GitHub Environment gate.
4. An approver clicks *Review deployments*, optionally adds a comment, and approves.
5. The `apply` job runs through the same checkout / Terraform / OIDC / Vault steps, then runs `terraform apply -auto-approve`. Output ends with:

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Manual approval is the load-bearing safety net

Because `apply` is `-auto-approve`, the GitHub Environment gate is the **only** human checkpoint. Treat the required-reviewers list with the same care as production deploy access for any other system. Two-person rule (author cannot self-approve) is the standard configuration in **Settings -> Environments -> production -> Prevent self-review**.

9. Step 7 — Validate

Three places to check, in order:

9.1 — Dynatrace tenant

1. Sign in to `https://<your-tenant>.apps.dynatrace.com`.
2. Open **Settings -> Cloud and virtualization -> Kubernetes -> Generic metadata enrichment**.
3. Confirm a rule exists with `sourceAttribute: namespace-label`, `sourceKey: lab-team`, `targetAttribute: dt.security_context`.
4. Open the rule's audit log entry — the **Modified by** column should show the service user (e.g. `svc-terraform-lab`), not a human.

9.2 — GitHub

1. **Actions** tab -> the merge-commit run -> confirm both jobs are green.
2. **Settings -> Environments -> production -> Deployment history** — confirm one deployment, with the approver's name.
3. The closed PR shows the *Terraform Plan* comment from the plan run.

9.3 — Vault audit log

```
# Tail the audit log if you have access
vault audit list
# Look for two reads in the last 10 minutes against
secret/data/dynatrace/lab,
# both authenticated via auth/jwt with role=dynatrace-terraform-lab.
```

The role's *Last vault token issued* metadata also updates — visible in the Vault UI under **Access -> Auth methods -> jwt -> Roles -> dynatrace-terraform-lab**.

If all three places show what you expect, the pipeline is wired correctly end-to-end.

10. Common Pitfalls

#	Symptom	Cause	Fix
1	Error: Could not satisfy plugin requirements on terraform init after a state-backend change	Stale .terraform/ cache	<code>rm -rf .terraform && terraform init</code>
2	Error: state lock could not be acquired	A previous run died holding the lock; with <code>use_lockfile = true</code> the lock object is <code>key.tflock</code> in the same prefix	Verify no run is actually in progress, then <code>terraform force-unlock <LOCK_ID></code> (the ID is in the error message)
3	Error: AccessDenied on s3:GetObject during terraform init	The AWS IAM role is right but the trust policy <code>sub</code> claim does not match the workflow context	Compare the OIDC <code>sub</code> printed by <code>actions/github-script</code> (debug step) against the role's trust policy <code>StringLike</code> condition; usually a typo in the org or repo name
4	permission denied: cannot retrieve credential from Vault	Workflow lacks <code>id-token: write</code> permission, or the Vault role's <code>bound_audiences</code> does not match <code>jwtGithubAudience</code>	Add <code>id-token: write</code> to the workflow's <code>permissions: block</code> ; align the audience strings exactly — Vault is whitespace and case sensitive on this field
5	Plan posts the Dynatrace token in cleartext into the PR comment	<code>terraform plan</code> rendered an attribute that contains the token (e.g. an output, or a debug <code>local</code>)	Mark any <code>output</code> referencing the token as <code>sensitive = true</code> and, more importantly, never echo <code>TF_VAR_*</code> or <code>DYNATRACE_PLATFORM_TOKEN</code> from a <code>run: step</code> — <code>vault-action</code> 's masking only catches values it injected, not values you re-stringify

#	Symptom	Cause	Fix
6	Error: 403 Forbidden from the Dynatrace API on apply	Three-things-align failure: service user IAM permissions, creator's <code>iam:service-</code> <code>users:use</code> at mint time, or scope selection	Re-read AUTOM-04 §3 <i>Service User Credentials</i> ; the most common variant is the service user lacking the Change monitoring settings permission
7	Plan on PR succeeds; same plan on merge- to-main fails	Drift between PR-time and merge-time tenant state, or the apply role can read state but the plan role cannot (or vice versa)	If you split the IAM roles per environment, both must permit S3 state access on the same prefix; for this LAB they share a role, so look for tenant drift first

11. Where to go next

Topic	Notebook
Other CI platforms (GitLab, Bitbucket, Bamboo, Azure DevOps) following the same OIDC + Vault shape	AUTOM-07 §4–§7
State backend hardening — KMS keys, IAM scopes per environment, multi-account state, audit logging	AUTOM-09 §3
Multi-environment promotion — <code>envs/dev/</code> , <code>envs/staging/</code> , <code>envs/prod/</code> directories vs Terraform workspaces	AUTOM-09 §4
Lifecycle protections — <code>prevent_destroy</code> , <code>ignore_changes</code> , <code>replace_triggered_by</code> for critical resources	AUTOM-09 §9
Token-model deep dive — Platform Token vs classic API Token vs OAuth Client; when each applies	AUTOM-04 §3
<code>dynatrace_api_token</code> / state-leakage operational safety	AUTOM-04 §3 <i>Operational Safety</i>
Drift detection (scheduled <code>terraform plan -detailed-exitcode</code> workflow)	AUTOM-07 §3 <i>Drift Detection</i>

Topic	Notebook
Reusable workflows for multi-team / multi-tenant orgs	AUTOM-07 §3 <i>Reusable Workflows</i>
Policy-as-code gates (OPA/Conftest, Sentinel)	AUTOM-07 §3 <i>Policy-as-Code Gates</i>

12. Summary Checklist

Tick off each item to confirm a working LAB:

- ☐ Repo created with `versions.tf`, `main.tf`, `backend.tf`, `.gitignore`
- ☐ S3 state bucket created, versioning enabled, encryption enabled
- ☐ Platform Token minted in DT UI on a service user; deposited in Vault at `secret/dynatrace/lab`
- ☐ AWS OIDC identity provider for `token.actions.githubusercontent.com` exists
- ☐ AWS IAM role `github-actions-dynatrace-terraform` with trust policy scoped to your repo
- ☐ Vault `jwt` auth method enabled and bound to GitHub's OIDC issuer
- ☐ Vault role `dynatrace-terraform-lab` with policy granting read on `secret/data/dynatrace/lab`
- ☐ `.github/workflows/terraform.yml` committed; permissions block contains `id-token: write`
- ☐ PR run shows green `plan` job; PR comment with plan summary posted
- ☐ No raw token values appear in any log line
- ☐ `production` GitHub Environment created; required reviewers configured; deployment branches restricted to `main`
- ☐ Merge-to-main run shows `apply` job waiting on approval; approval triggers apply
- ☐ Validation: DT tenant shows the new Settings 2.0 rule; modified-by = service user
- ☐ Validation: GitHub Environments page shows the deployment with approver's name
- ☐ Validation: Vault audit log shows two reads from the workflow's OIDC identity

13. Sources and References

Dynatrace

- [Platform Tokens \(DT docs\)](#)
- [Terraform for Dynatrace \(DT docs\)](#)
- [Dynatrace Terraform Provider \(Terraform Registry\)](#)
- [Dynatrace Terraform Provider \(Dynatrace GitHub\)](#)

GitHub Actions

- [About security hardening with OpenID Connect \(GitHub docs\)](#)
- [Configuring OpenID Connect in Amazon Web Services \(GitHub docs\)](#)
- [Using secrets in GitHub Actions \(GitHub docs\)](#)
- [Using environments for deployment \(GitHub docs\)](#)

Actions used in the workflow

- [hashicorp/setup-terraform \(HashiCorp GitHub\)](#)
- [hashicorp/vault-action \(HashiCorp GitHub\)](#)
- [aws-actions/configure-aws-credentials \(AWS GitHub\)](#)
- [actions/github-script \(GitHub\)](#)

Terraform / Vault

- [S3 backend \(Terraform docs\)](#) — current S3-lockfile locking model
- [Sensitive data in state \(Terraform docs\)](#)
- [JWT auth method \(Vault docs\)](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.